Deployment Guide

Generated 2026-10-11 from `docs/deployment-guide.md`

LAN deployment for Bukidnon Cable and Internet Services' three office PCs, per the lab spec's required architecture (§2.1):

```
PC 1: Owner/Admin Desktop   ─┐
PC 2: Cashier Desktop        ├── LAN ──> BCIS API Server ──> PostgreSQL
PC 3: Operations Desktop    ─┘
```

One machine on the office LAN runs the API server and PostgreSQL (this can be one of the three office PCs, or a small dedicated machine — the spec does not require a separate server box). All three desktop clients point at that machine's LAN IP address; none of them opens PostgreSQL directly.

1. Server Machine Prerequisites

- Windows 10/11 (or any OS PostgreSQL + Node.js run on).
- Node.js 20+ and npm.
- PostgreSQL 16+ (or XAMPP's bundled PostgreSQL / a separate install).
- A static LAN IP, or a DHCP reservation, so client machines' `.env` files don't need updating every time the server reboots.
- `pg_dump` and `pg_restore` (installed alongside PostgreSQL) on the PATH — required for Backup & Restore (`api/src/lib/backup.ts` shells out to them).

2. Database Setup

```
# As the postgres superuser, or a role with CREATEDB:
createdb bcis_billing
```

Never point this at an existing production database — the lab spec requires synthetic data only (§8, Data Privacy).

3. API Server Setup

```
cd api
cp .env.example .env
```

Edit `.env`:

```
DATABASE_URL=postgres://<user>:<password>@localhost:5432/bcis_billing
JWT_SECRET=<a long random string — generate one, don't reuse this example>
PORT=3000
HOST=0.0.0.0          # must be 0.0.0.0, not localhost, so LAN clients can reach it
BACKUP_DIR=./backups  # outside the web root; the API never serves this directory
```

```
npm install
npm run db:migrate          # applies every migration under api/drizzle/ to a clean database
npm run db:seed             # roles + a full synthetic demo dataset (never real subscriber data)
npm run bootstrap:owner     # one-time: creates the first Owner login (see below)
npm run dev                 # or: npm run build && npm start for a non-watch process
```

`bootstrap:owner` needs four environment variables (pass them inline or add to `.env` temporarily, then remove them):

```
BOOTSTRAP_OWNER_USERNAME=admin \
BOOTSTRAP_OWNER_EMAIL=admin@example.test \
BOOTSTRAP_OWNER_FULL_NAME="System Owner" \
BOOTSTRAP_OWNER_PASSWORD=<at least 12 characters> \
npm run bootstrap:owner
```

Confirm the server is reachable from another machine on the LAN:

```
curl http://<server-lan-ip>:3000/health
```

Running the API as a persistent service

For a real office deployment (not just a dev session), run the API under a process manager so it survives reboots/crashes, e.g.:

```
npm run build
npx pm2 start dist/index.js --name bcis-api
npx pm2 save
npx pm2 startup   # follow the printed instructions to run pm2 at boot
```

(Any Windows service wrapper — NSSM, Task Scheduler at logon, `pm2-windows-startup` — works equally well; the specific choice isn't prescribed by the spec.)

4. Desktop Client Setup (each of the three PCs)

```
cd desktop
cp .env.example .env
```

Each client needs the server's LAN address. After installing, open the app and on the login screen click **Server: … Change**, enter `http://<server-lan-ip>:3000`, then **Test & Save**. The address is health-checked before it is saved and is remembered on that PC, so no file editing is needed on installed clients. (`VITE_API_BASE_URL` in `desktop/.env` is still honoured as the default when running `npm run dev`.)

Plain `http://` is accepted only for this PC and private office-LAN addresses (10.x, 172.16–31.x, 192.168.x, `*.local`); any other address must use `https://`.

```
npm install
npm run dev     # development mode
# or, to build an installer:
npm run dist:win
```

`npm run dist:win` produces `desktop/release/BCIS Billing-Setup-<version>.exe` (NSIS installer, unsigned — no code-signing certificate is configured) and an unpacked copy at `desktop/release/win-unpacked/`. Run the installer on each client machine; it offers a destination folder and desktop/Start Menu shortcuts. Set the server address from the login screen on first launch (see above).

> **Windows `file://` gotcha, already fixed**: a packaged Electron + Vite + React Router app can render a permanently blank window on Windows even though `npm run dev` works perfectly — two independent causes, both present in this project until they were found and fixed: (1) Vite's default absolute asset base (`/assets/...`) resolves against the filesystem root instead of the app directory when the page is loaded via `file://`, and (2) `BrowserRouter`'s History-API routing corrupts `location.href` to the drive root the moment it mounts under `file://`. Fixed via `base: "./"` in `electron.vite.config.ts` and switching to `HashRouter` in `App.tsx`. If this stack is ever changed, re-verify by running `npm run dist:win` and launching the actual `release/win-unpacked/BCIS Billing.exe` — not just `npm run dev`, which never exercises this code path.

5. Verifying Three-Client Concurrency (AT-09)

With the API running and at least two desktop clients pointed at it:

1. Log in as different users on each client (e.g. a Cashier on PC 2, an Administrator on PC 1).
2. From PC 2, post a payment against one service account while PC 1 generates an invoice for a *different* service account at roughly the same time.
3. Confirm on both clients: no duplicate invoice/receipt numbers, no error, and each client's data refreshes to the correct shared state. The underlying guarantee is enforced by PostgreSQL transactions and unique constraints, not client-side coordination — see `api/tests/payments.test.ts` ("concurrency" describe block) and `api/tests/invoices.test.ts` ("concurrent duplicate creation…") for the automated version of this same scenario.

6. Backup Schedule

Run a backup after each business day, at minimum:

- **Manual**: Owner logs into any client → **Administration** → **Backup & Restore** → **Create Backup**.
- **Scheduled** (recommended for production use): a scheduled task on the server machine calling `POST /api/v1/backup` with an Owner-role bearer token, or a cron/Task Scheduler job running `pg_dump` directly into `BACKUP_DIR` and letting the next manual **Create Backup** pick up the `backup_history` bookkeeping. Verify a backup periodically with **Verify** (non-destructive) and test a full restore against a scratch database on a non-production schedule — see `api/tests/backup.test.ts` for exactly that cycle, automated.

7. Firewall / Network Notes

- Open inbound TCP port 3000 (or your configured `PORT`) on the server machine's firewall for the LAN subnet only.
- PostgreSQL's own port (5432 by default) should **not** be exposed beyond the server machine itself — only the API process needs to reach it (`DATABASE_URL` uses `localhost` from the API server's own point of view).

8. Updating a Deployed Instance

```
cd api && git pull && npm install && npm run db:migrate && npm run build
# restart the API process (pm2 restart bcis-api, or equivalent)

cd ../desktop && git pull && npm install && npm run build
# redistribute the updated build to each client (npm run dev during
# development, or a rebuilt installer once packaging is configured)
```

Migrations are additive and run via `npm run db:migrate` — never hand-edit the schema or run `drizzle-kit push` against a database with real data.